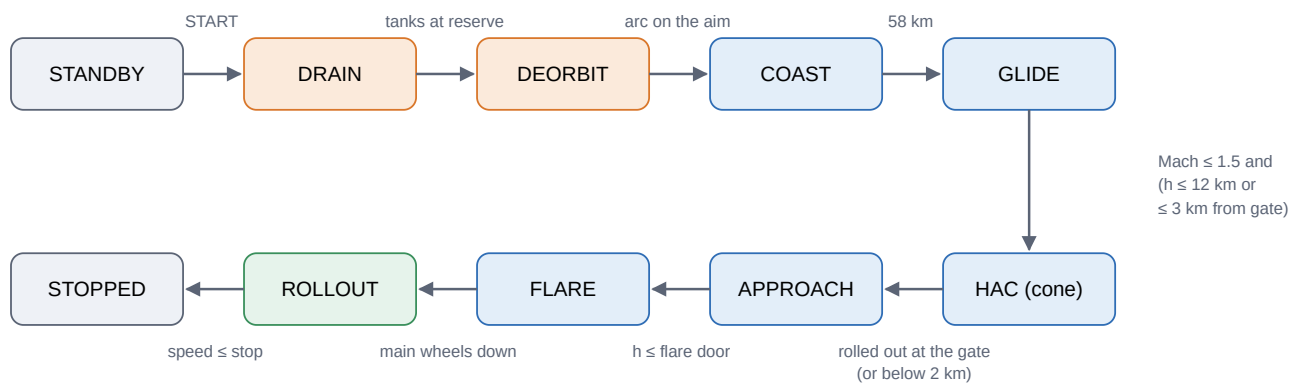


How the spaceplane autopilot works

krpcLab · spaceplane · what the program does and what makes it change what it is doing. State of the code on 2026-10-09.

The autopilot takes a winged vehicle (the target is the Mk3 shuttle `qs_shuttle2`) from orbit to a stop on the KSC runway. It does not follow a stored plan. Every tick it reads the vehicle's state, **propagates the rest of the flight to the ground** with an RK4 integrator (`trajectory.py`), and takes each decision from that fresh prediction. `autopilot.py` is the phase machine and the only code that talks to the game; `guidance.py` holds the control laws as pure functions; `environment.py` holds the aerodynamic table the vehicle measures for itself; every tunable is in `config.py`.

The phase machine



Engaged below the interface, START skips to GLIDE, APPROACH or FLARE by height and distance to the runway.

Figure 1. Phases and the conditions that advance them. Orange phases use the engine; blue are unpowered flight; green is on the ground.

Things that run in every phase

mechanism	what it does, and when
Prediction	A propagation to the ground of the law the vehicle will fly (the same function the controller calls), from the current state, each tick. The glide and the deorbit solve against it; the cone and approach plan from it.
Aerodynamic table	Swept in STANDBY with kRPC's <code>simulate_aerodynamic_force_at</code> over angle of attack and Mach; rows are re-probed in flight (a few per second). Stall speed, best L/D and approach speeds are read from it, not configured.
Attitude	kRPC's autopilot points the nose at the commanded (alpha, bank). Its response times are derived from the vessel's inertia and torque, retuned when surface authority grows with dynamic pressure. A roll damper lengthens roll response when bank swings grow and recovers it after.
RCS valve	Open when the pointing error exceeds 5°, closed 2 s after it is under 1.5°. Allowed in the burn, the coast, the glide above Mach 1 and during bank reversals.
Alpha ceiling	Learned: if the vehicle sits well below its commanded alpha the ceiling ratchets down to what it holds, and the prediction uses the same limit (<code>trajectory.Holdable</code> , binned by dynamic pressure).
Pitch trim	Above Mach 3 in the glide, the standing pitch input is moved off kRPC's integrators into a body-frame manual trim (<code>GLIDE_PITCH_OFFLOAD</code>). In the cone and approach the two nose canards are taken off kRPC and driven as a trim that walks the standing pitch input to zero (<code>CANARD_TRIM</code> , under 15° alpha, settled).
Time scale	The farm's time-scale plugin is set so each tick's game interval stays what the phase asked for; every log ends with a <code>loop rate</code> line.

Phase by phase

STANDBY — until START is pressed

Configures the craft (main gear brakes 200% / friction 10, nose gear brake off), measures wheel clearance and tail-strike angle, tunes the attitude controller, sweeps the aerodynamic table, and probes the brakes it can build from the surfaces (split rudder, opposed flaps). On START: below the 58 km interface → GLIDE / APPROACH / FLARE by height and distance; otherwise → DRAIN.

DRAIN — before the burn

Opens the craft's drain valves and empties propellant down to what the burn needs: the dv a deorbit can take ($200 \text{ m/s} \times 1.25 \text{ margin}$) at the engine's vacuum Isp. Closes the valves, → DEORBIT. Doing it before the burn keeps every mass change upstream of the burn's own measurement.

DEORBIT — wait for a pass, then one burn

Each tick (2 s): choose the runway end; if the gate is 400–2300 km of arc ahead, search for a burn.

- **Search** (`deorbit_solution`): a grid then refinement over retrograde dv 10–400 m/s. For each candidate it propagates the two corners of what the glide may later command (`deorbit_window`): the shortest entry (high drag) and the longest (most lift). The burn chosen puts the gate an eighth of the window's width short of its centre (`DEORBIT_WINDOW_BIAS` 0.25), the side the glide can always correct by shortening.
- **No solution**: rails-warp to the next sample (the warp step is learned from how much orbit each warped tick covers) and ask again.
- **Burn**: point retrograde (RCS allowed), limit thrust so the burn lasts at least 3 s, throttle tapered on the dv still owed. The stop test re-propagates the current arc each tick and measures the remaining dv against the chosen aim; when that propagation cannot answer (arrival past the 1500 s horizon), the burn stops on the solved dv and its exit line says *unmeasured*. → COAST.

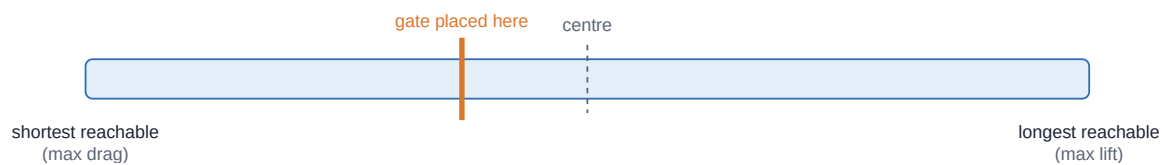


Figure 2. The deorbit window. The burn's dv moves the whole window; it is chosen so the gate sits 25% of the half-width short of centre.

COAST — to the 58 km interface

Swings to the entry attitude (22° alpha, 30° bank toward the runway) in vacuum, where holding it costs nothing, and rails-warps the part above the atmosphere (released 2 km above it). The bank's side is latched so the glide starts on the same lean. At 58 km → GLIDE. `COAST_WATCH_S` (off) logs the burn's stop test through the coast.

GLIDE — the entry, Mach 25 to Mach 1.5

Each tick (1 s): solve alpha and bank so the predicted arc crosses 12 km over the gate (+ a 500 m reserve past it).

control	role
alpha (20–40°)	Downrange: more alpha, more drag. Solved against the along-track miss, rate-limited to 3°/s, held still during a bank reversal, and clamped by speed (a cap holds the approach speed near the end) and by the learned ceiling.
bank magnitude (30–70°)	Takes what alpha leaves: rolling lift off vertical sinks the vehicle into denser air. When both are at their stops the solve returns the longest/shortest arc it can reach ($s_{lv} = \max$).
bank sign	Cross-track: lean toward the runway; reverse when the predicted cross-track leaves a deadband of 20 m per km to run (500 m–40 km). The reversal commits once the bank has passed through 30° the other way. Bank slews at a rate measured from the vehicle's roll.

At Mach ≤ 1.5 and either 12 km over the field or within 3 km of the gate: release the alpha ceiling for the landing, pick the runway end and turn direction that are cheaper from here (`hac_choose`) → HAC.

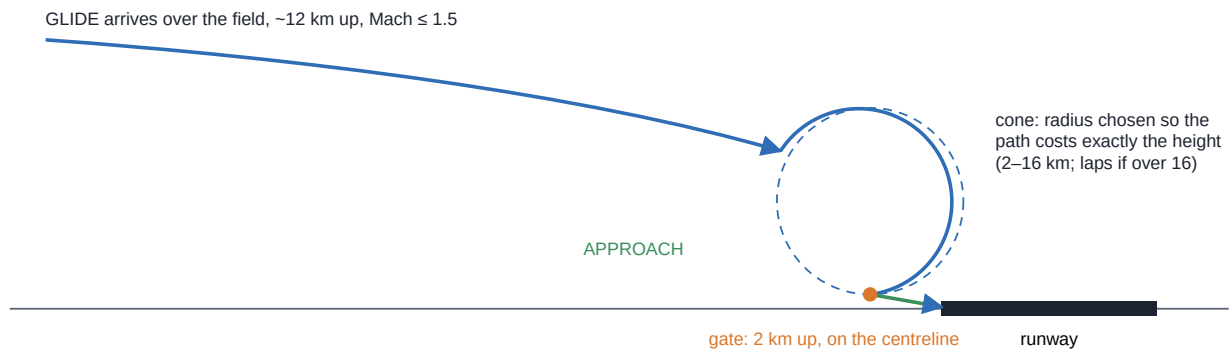


Figure 3. The heading-alignment cone. Whatever energy the entry delivers becomes a turn radius; the rollout is the gate.

HAC — the heading-alignment cone

Each tick: re-plan the circle from the present state (`guidance.hac`).

- **Radius:** the circle (tangent to the extended centreline, ending at the gate) whose path, priced down a ladder of 500 m slices at the glide ratio the vehicle flies there (the table's shape, scaled by the L/D measured in flight), spends the height above the gate. 2–16 km; past 16 km a full lap is added. The radius moves at a limited rate.
- **Speed and alpha:** hold the cone's target speed (the approach speed, from the measured stall) with alpha up to 22°.
- **Bank:** the bank that flies the circle, capped at 45°, plus a **weave** (up to 50° off track, starting on the side the vehicle is already banked) when there is more than 800 m of surplus height.
- **Exit:** lined up (within 12°), within 900 m of the gate, no lap left, and no more than the approach can absorb above its need → APPROACH. Below the gate's 2 km → APPROACH regardless.

APPROACH — final, 2 km to the flare

Each tick (0.1 s): `guidance.approach` plus the split brake.

- **Aim point:** 1800 m past the threshold (the approach aims 1000 m nearer, so the flare's float lands in the zone).
- **Speed, not path:** alpha is set from the descent angle that holds the target speed ($2.25\times$ stall), from $dv/dt = g \sin\theta - D/m$, so it can make speed by steepening as well as lose it.
- **Lateral:** a closure that brings cross-track to zero as the wheels arrive; **S-turns** (up to 45° of track) when more than 200 m of excess height remains and the aim is over 4 km away.
- **Split-rudder speedbrake** (`APPROACH_SPLIT_BRAKE`): the two vertical fins deploy in opposite directions (no yaw or roll; up to 38° cuts L/D by ~55%). Each tick it opens to the L/D factor the geometry asks for, $distance / (height \times best\ L/D)$, at 10°/s. It stows when the vehicle is slower than $1.45\times$ stall, when sinking faster than $1.5\times$ the design sink, and 4 s of sink before the flare door.
- **Flare door:** 50 m + $2.5\ s \times$ sink rate. At or below it → FLARE.

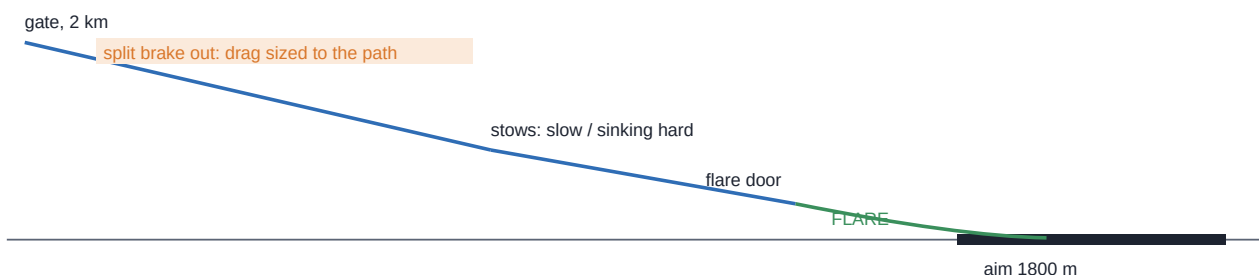


Figure 4. Final approach: the brake buys path angle while alpha holds speed; it is in before the flare.

FLARE — the last 50–100 m

Stows the speedbrake. Tracks a sink rate of $\sqrt{(v_{td}^2 + 2 a h)}$ that eases to the touchdown rate, with alpha capped by maximum lift (30°) and by the tail-strike angle. Above 60 m it may bank up to 12° (tapering to wings level) to close cross-track; below 30 m the nose is aligned with the *runway*, not the airflow. A proportional pitch term (0.08 per degree) helps kRPC hold the flare attitude. Gear down. Main wheels touching → ROLLOUT.

ROLLOUT — on the runway

Ground spoiler out on main-gear contact. The nose is ramped down from its touchdown attitude over 1.5 s (never stepped), the trims ramp out, full main-gear braking, nosewheel steering onto the centreline on cross-track plus its rate times a 3 s lead. At the stop speed → STOPPED, which logs the stop position and hands the vessel back.

Which law decides what

quantity	GLIDE	HAC	APPROACH	FLARE / ROLLOUT
downrange / energy	alpha, then bank magnitude	circle radius, weave, laps	split brake, S-turns	—
speed	speed cap on alpha (late)	alpha to the target speed	descent angle that holds speed	sink-rate law / brakes
cross-track	bank sign, deadband	circle geometry	closure, wheels-time	bank ≤12°, then nosewheel
pitch trim	body-frame offload (Mach > 3)	canard trim		P-term in the flare, ramped out

Logs

Nothing prints. Each run writes `logs/LOG<n>`: a `config:` line (fields different from the defaults, plus the defaults fingerprint), one telemetry line per tick interval, every phase change and decision as an event, and a closing `loop rate` line. `spaceplane/tools/` reads them (`landsum.py`, `armsum.py`, `conesum.py`, `osccsum.py`).